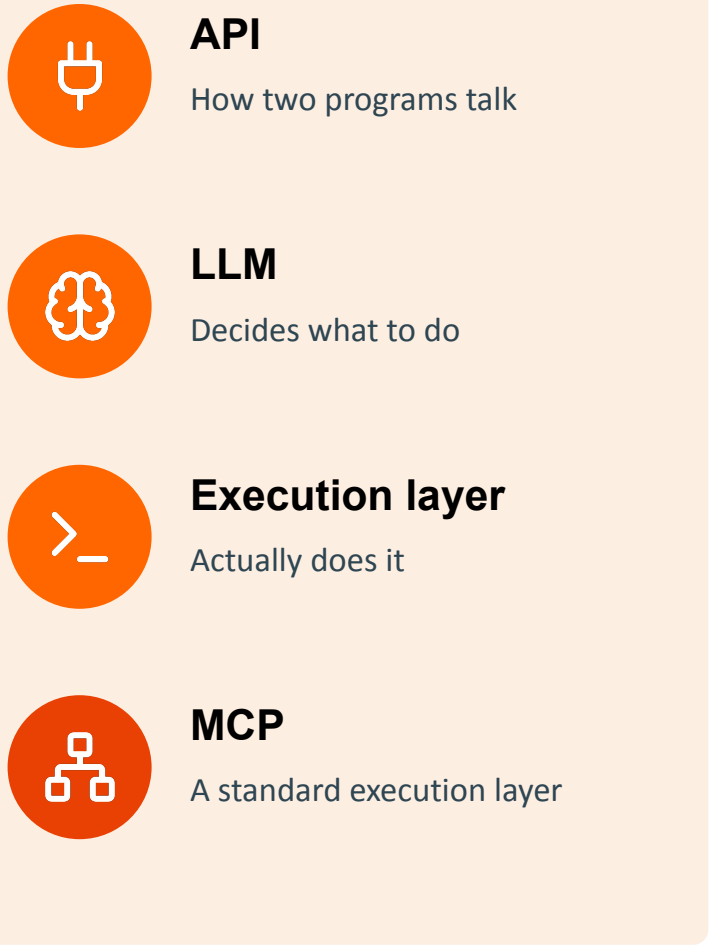
MCP vs API: Settling It Properly

What MCP actually is, where it sits in your application, how it compares with APIs — and the confusions people usually have about it

MCP = Model Context Protocol



unext

THE DEBATE

“MCP Changes Everything” vs “We Already Have APIs”

Camp 1

“MCP changes everything.”

Camp 2

“We already have APIs. Why a new thing?”

Both are partly right. Here is the path through it:

1

APIs, then APIs + an LLM

How apps call services today — and what really changes when a model joins

2

What MCP is and where it sits

Clients, servers, discovery, and one full round trip

3

Why not just the OpenAPI spec?

The execution layer, a side-by-side Slack task, and the real differences

4

Warnings and when to use which

Context cost, API keys, and a one-line rule you can repeat

START WITH WHAT WE KNOW

An API Is How Two Programs Talk to Each Other

Program A needs to read messages from Slack (program B). Slack exposes an endpoint — just a URL.



Clean and predictable — perfect for normal software.

The important reality: you, the developer, build each integration — and write the rules for when to call it

IF

When a deploy finishes

→ call Slack's send-message endpoint

IF

When someone asks for yesterday's messages

→ call the history endpoint

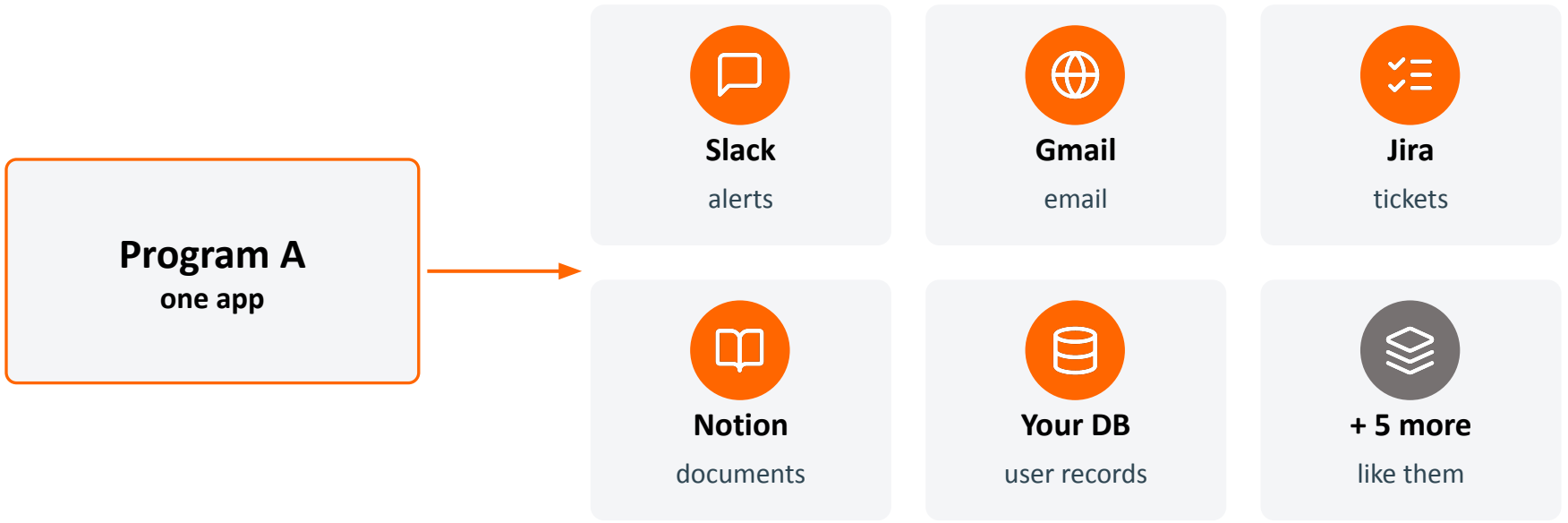
IF

If this happens / if that happens

→ call Slack / call Jira

One App Rarely Talks to Just One Service

A typical app talks to ~10 services — and every one needs the same plumbing written by hand.



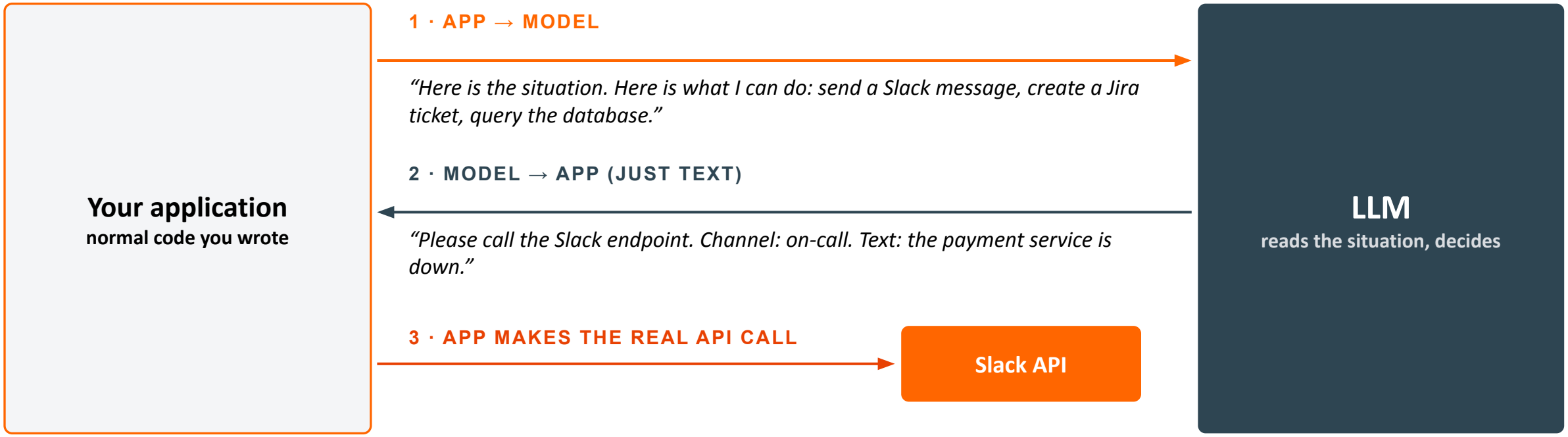
Per service, per app

- Authentication
- The API calls
- Error handling
- Retries when a call fails

For each of the 10 services, you write the same kind of logic — for every app that needs it.

Add an LLM: The Model Decides, the App Executes

The secret that makes everything else easier: the model does not make the API call. It only writes what it wants.



Nothing changed underneath: you still wrote all 10 integrations by hand. The model now chooses which call to make — but you built every one.

Second App, Same 10 Services — Written All Over Again

Your org adds a customer-support chat app, also LLM-powered. It needs Gmail, Jira and the rest — again.

10

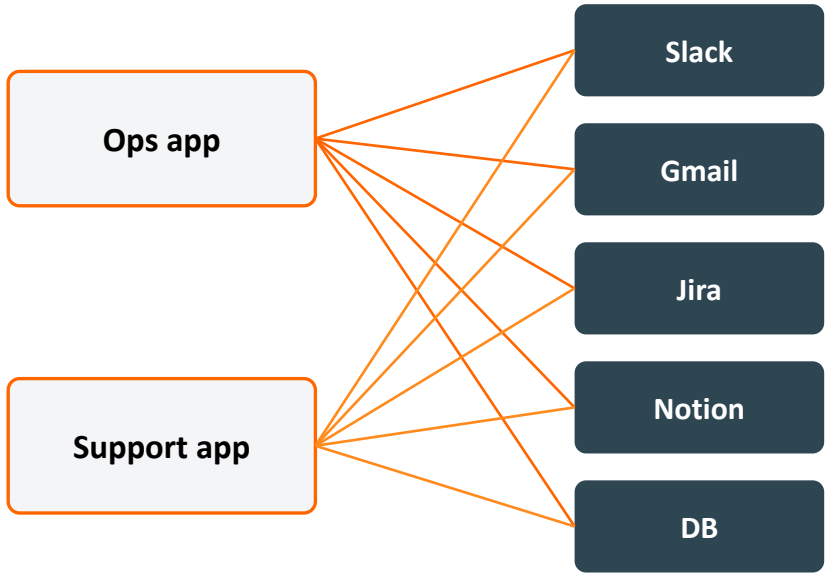
integrations for app 1

20

for two apps × 10 services

30

when app 3 arrives next year

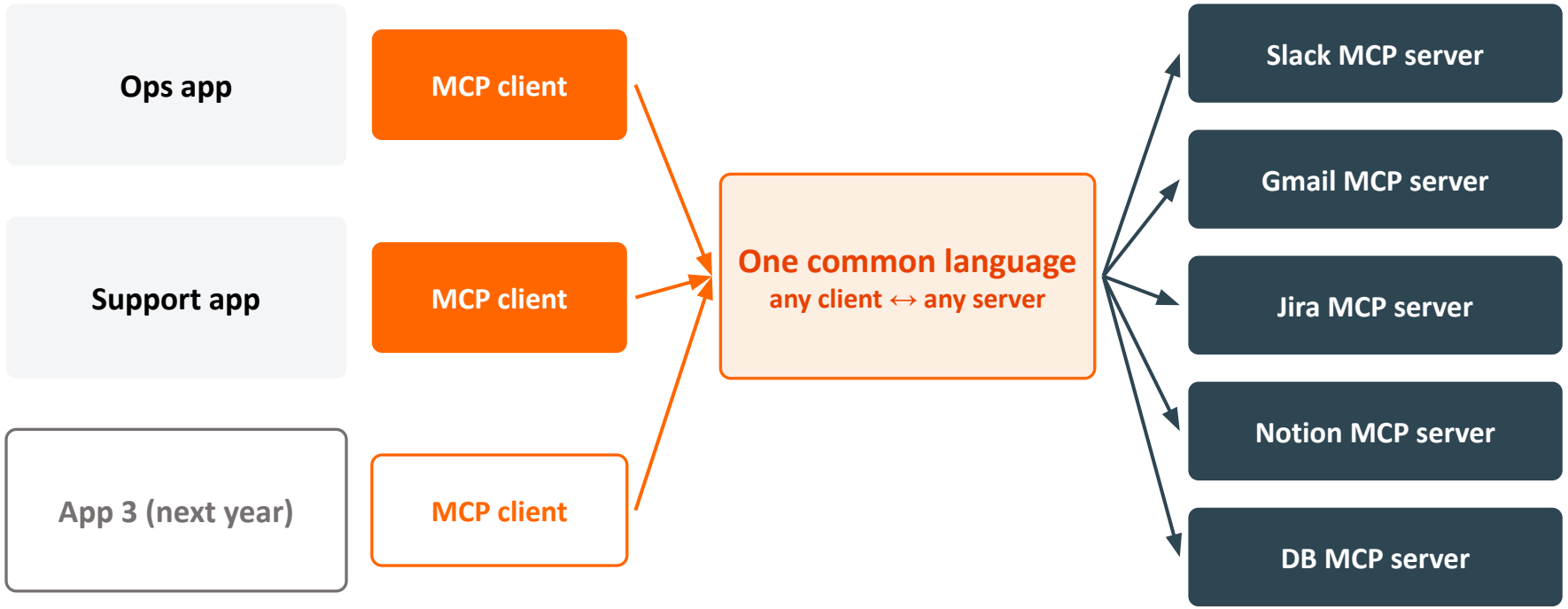


Every line is integration code to write and maintain

The question that led to MCP

What if nobody had to hardcode these API calls into every app?

One Client per App, One Server per Service



2 + 10

≈ 12 pieces instead of 20 separate integrations

App 3 next year?

Reuses the same 10 servers — add one client.

Popular products

The server often already exists — Slack builds and hosts its own official MCP server.

Your DB and internal tools

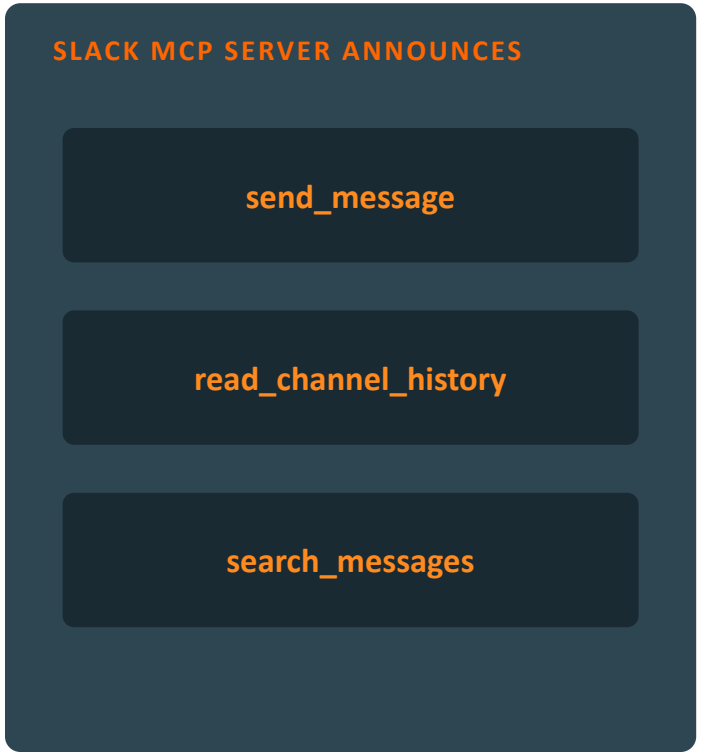
Your team may still need to build that server yourself.

The client

Most teams take it from an AI framework instead of writing it from scratch.

Discovery: The Model Picks From a Labelled Toolbox

This is the part people get wrong. The client asks each server what it can do; the app shows that menu to the model.



1 · Discover

MCP client connects to each server and asks: what actions are available?

2 · Show the menu

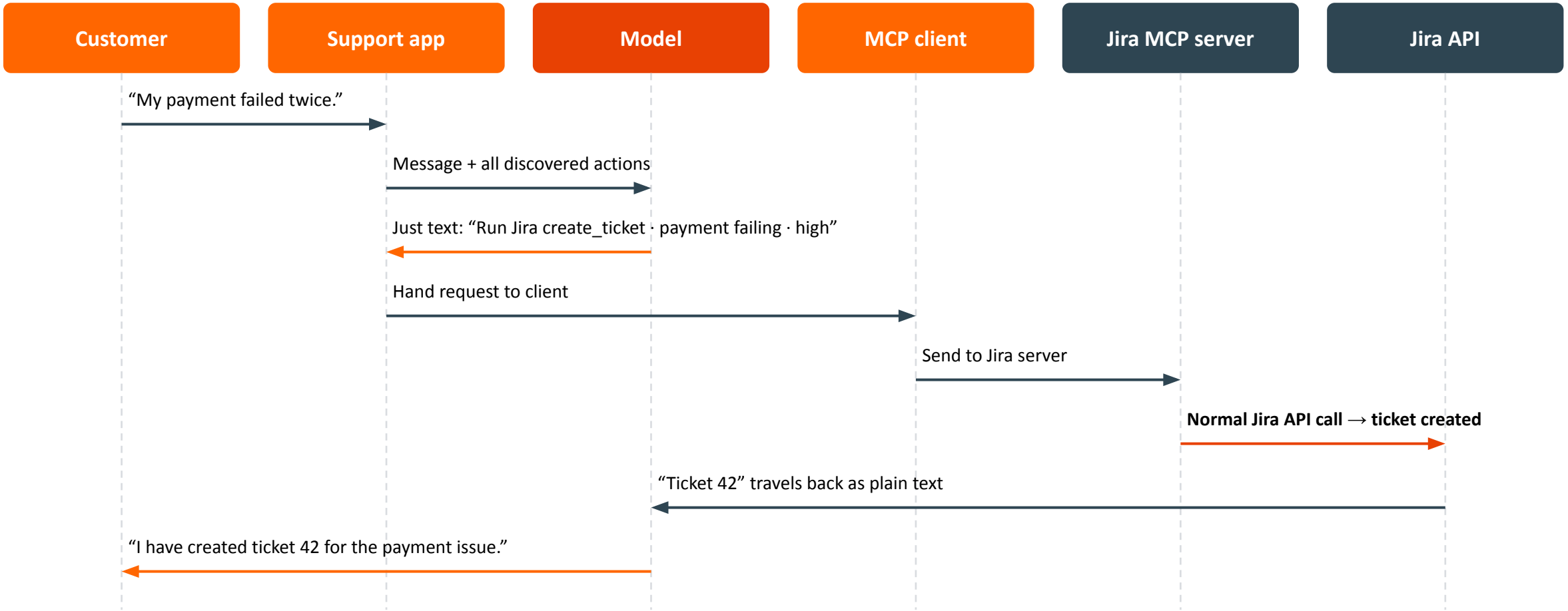
The app passes that list of actions to the model, with the task.

3 · Model chooses

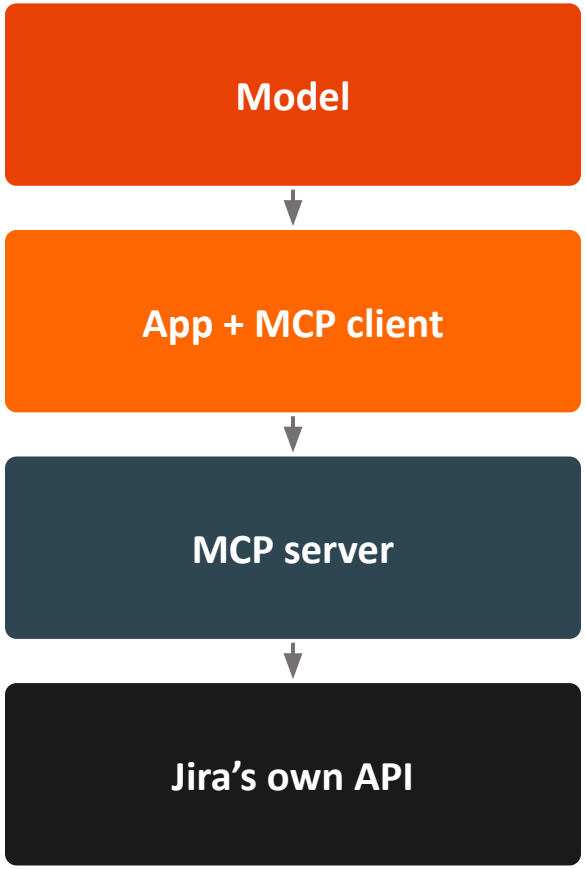
The model picks the action that fits — only from what the app allows.

You no longer hardcode every decision — you hand the model a labelled toolbox and let it pick.

“My Payment Failed Twice” — No Magic Left Anywhere



No. MCP Is an AI-Facing Layer in Front of Your APIs



Only decided and replied

Carried the request

Translated into API calls

Did the real work — exactly as always

The translator analogy

- If a system already has an API, the MCP server sits in front of it as a translator
- Your API stays exactly where it is
- MCP is not a replacement for APIs — it is a common AI-facing layer on top of them

PART 2 · THE CONFUSIONS

“Why Not Just Give the Agent the OpenAPI Spec?”

A fair question — it works. So let’s give both approaches the exact same task and see precisely where MCP adds something.

unext

The Execution Layer: Where Actions Really Happen

The model cannot make an API call. Something outside it must pick up the request and do it.



Both approaches we compare are just two ways of building this layer:

Approach 1 · Direct API

API docs + a sandbox. Custom code — partly written by the agent itself — is the execution layer.

Approach 2 · MCP

An MCP server in front of the service. The server is the execution layer.

SAME TASK, TWO WAYS

Summarise Yesterday's #incident Messages

A human does this in a minute: open Slack, find the channel, read, summarise.

Approach 1 · Direct API

Agent gets Slack's full API docs + a sandbox with credentials

- Reads docs for hundreds of API methods
- Writes Python that chains several calls:
  - look up #incident → channel ID
  - call message-history API
  - convert "yesterday" to Slack's time format
  - resolve raw user IDs to names
- Sandbox runs it → clean messages → summary

Works. But your team sets up credentials, runs the sandbox, and fixes it when it breaks.

Approach 2 · MCP

No docs — the app connects to a Slack MCP server

search_channel	**read_messages**
search_by_date	**send_message**

Model picks read_messages. The server handles:

channel-ID lookup · time formats · pagination · authentication

Works too. The model never touches a low-level Slack method; all Slack knowledge lives in one reusable piece.

A Valid Design — You Just Reinvented MCP Privately

What you had to invent

- A format for describing actions
- A format for sending requests
- A format for returning results
- Client code in every app that speaks it

What MCP is

- The same idea — but the format is public
- Compatible AI applications already speak it
- Servers and clients already exist for many tools

Analogy: why design your own database when ready-made solutions like Postgres already exist?

Three Real Differences

1

It educates — it doesn't execute

An OpenAPI spec tells the agent which calls exist. You still need software that holds credentials, makes the request, handles failures and returns the result. MCP gives you a standard execution layer.

2

Written for developers, not models

Raw methods are granular steps developers chain together. An MCP server wraps them into one clearly named action — and when Slack changes a method, the hosted server is fixed in one place.

3

Not everything is a web API

OpenAPI only describes web APIs. Your local files, your terminal, your database have no OpenAPI document — but all of them can have an MCP server.

MCP Does Not Solve Everything

Context cost

- Big API docs can fill the context window and burn tokens
- MCP has the same problem if the app loads every action from every server
- Newer AI apps search for the right actions and load only what the task needs

API keys

- API keys are passwords — the model should never see them
- The execution layer outside the model stores the keys and connects to the service
- MCP supports login, permissions and approval before risky actions — but developers must set them up correctly

DECISION GUIDE

When to Use a Direct Integration vs MCP

Direct integration is reasonable when…

- One application, a few API operations you already know
- A quick experiment or a one-off task
- No useful MCP server exists yet — or it lacks the action you need
- Plenty of teams run secure, reusable direct integrations — that's fine

MCP pays for itself when…

- Several AI applications need the same systems
- You want capabilities discoverable and reusable across apps
- Non-web systems (files, terminal, DB) need an AI-facing layer
- You'd rather adopt a standard than maintain a private one

Remember: MCP gives the model no new abilities. Underneath, it's still normal APIs and normal code. What it gives you is a standard — so 10 apps share one execution layer instead of building 10.

THE ONE-LINER FOR NEXT TIME

The Model Never Executes the API Call

The execution layer around it does. Build that layer yourself for one application — or use MCP when you want a standard layer that many AI applications can discover and reuse.

unext

Thank You

unext